



FIRST SEMESTER, 2026–2027
Course Handout Part II

Date: 1st August 2026

In addition to part I (General Handout for all courses appended to the Timetable), this portion gives further specific details regarding the course.

Course No : BITS F232
Course Title : Foundations of Data Structures and Algorithms
Instructor-in-Charge : Veena Prabhakaran (veena.prabhakaran@pilani.bits-pilani.ac.in)
Lab Instructors : Veena Prabhakaran (veena.prabhakaran@pilani.bits-pilani.ac.in),
Neha Sangwan (p20240031@pilani.bits-pilani.ac.in)

1. Course Description:

Algorithm Analysis – Mathematical preliminaries, Sorting Algorithms, Search Algorithms, Linear Structures, Non-Linear Structures, Hashing, Graphs and Algorithms.

(Open elective for non-B.E. Computer Science students)

2. Scope & Objective of the Course:

A data structure is a way to organize large amounts of data, their relationships, and the operations that can be applied to them. This course focuses on applying data structures like stacks, queues, linked lists, heaps, trees, and graphs, along with algorithmic techniques such as brute-force, greedy, divide-and-conquer, and dynamic programming, to solve real-world problems, including shortest paths in maps, spell checking, and social network analysis.

Objectives

- Apply basic data structures (stacks, queues, linked lists, trees) to solve programming problems.
- Implement advanced data structures (graphs, balanced search trees, hash tables, priority queues) for complex applications.
- Use algorithmic techniques (brute-force, greedy, divide-and-conquer, dynamic programming) to address real-world challenges.
- Develop skills to design and implement data abstractions for problem-solving.

3. Course learning outcomes

- CLO1.** Apply appropriate data structures such as arrays, linked lists, stacks, queues, trees, graphs, and hash tables to solve computational problems.
- CLO2.** Perform a basic analysis and comparison of algorithms based on correctness, efficiency, and suitability for different problem domains.

CLO3. Design and implement solutions using fundamental algorithmic paradigms, including divide-and-conquer, greedy, and dynamic programming.

CLO4. Develop efficient programs by selecting suitable data structures and algorithms for real-world applications.

4. Textbooks and References

Textbooks:

TB1. *Introduction to Algorithms*, Cormen, T.H., Leiserson, C.E., Rivest, R.L., and Stein, C. 2nd Edition. MIT Press (Indian reprint: Prentice-Hall, 2009).

References:

Ref1. *Data Structures and Algorithm Analysis in C*, Mark Allen Weiss, 2nd edition, Pearson Education India; 2002.

Ref2. *Algorithm Design: Foundations, Analysis and Internet Examples*, Goodrich, M.T., and Tamassia, R. John, Wiley & Sons, Inc., 2002.

Ref3. *Algorithm Design*, Kleinberg, J., and Tardos, E., Pearson Education, 2007.

Ref4. *Algorithms* DasGupta, S., Papadimitriou, C., and Vazirani, U., Tata McGraw-Hill Publishers.

5. Course Plan:

5.1 Lecture Schedule

The course consists of 40 lectures (50 minutes each), covering data structures, algorithms, and their applications. The following table outlines the topics, learning outcomes, and readings.

Lecture no.	Description	Learning Outcomes	Reference
1	Course overview: Importance of data structures and algorithms, real-world examples (e.g., web browsing, GPS navigation)	Understand the role of data structures and algorithms in solving practical problems like web browsing and navigation systems	TB1: Ch 1
2	Algorithms: Definition, basic efficiency (fast vs. slow), Asymptotic notations, example: linear search	Recognize algorithms as step-by-step solutions, compare different functions in terms of asymptotic notations and apply linear search to find elements in datasets	TB1: Ch 1-2
3	Arrays: Storing data, search, insert, delete, applications (e.g., contact lists)	Implement arrays to store and manipulate data, enabling applications like contact list management	Class Notes
Continued on next page			

Lecture no.	Description	Learning Outcomes	Reference
4	Recursion: Basics, recursive vs. iterative, example: factorial, Solving recurrence relations	Write recursive algorithms to solve problems like factorial computation, comparing with iterative approaches	Class Notes
5	Divide-and-conquer, Searching: Concept, binary search, applications (e.g., searching contacts)	Apply divide-and-conquer to optimize searches, implementing binary search for efficient contact lookup	TB1: Ch 27 Pg: 799
6-7	Sorting: Bubble Sort, Insertion sort, intuition, applications (e.g., sorting playlists)	Implement insertion sort to organize small datasets, such as sorting playlists by title or artist	TB1: Ch 2, Ref1: Ch 7
8-9	Sorting: Merge sort, applications (e.g., large datasets)	Implement merge sort to handle large datasets, enabling applications like sorting extensive user records	TB1: Ch 2 Ref1: Ch 7
10-11	Sorting: Quick sort, pivot selection, applications (e.g., database queries)	Implement quick sort with effective pivot selection for fast sorting in database query optimization	TB1: Ch 2 Ref1: Ch 7
12	Sorting: Brief overview of non-comparison sorting algorithms (Counting, Radix, and Bucket Sort)	Identify the characteristics and applications of non-comparison sorting algorithms.	TB1: Ch 8
13-14	Linked Lists: Singly linked lists, insert, delete, applications (e.g., browser history)	Implement singly linked lists to manage dynamic data, supporting applications like browser history navigation	TB1: Ch 10 Ref1: Ch 3
15	Linked Lists: Doubly linked lists, advantages over singly linked lists	Understand trade-offs between singly and doubly linked lists for bidirectional data access	TB1: Ch 10 Ref1: Ch 3
16-17	Stacks: Stack ADT, array vs. linked list, applications (e.g., undo in editors)	Implement stacks using arrays or linked lists to solve LIFO problems like undo functionality in text editors	TB1: Ch 10 Ref1: Ch 3
18-19	C Queues: Queue ADT, array vs. linked list, applications (e.g., task scheduling)	Implement queues using arrays or linked lists for FIFO problems like task scheduling in operating systems	TB1: Ch 10 Ref1: Ch 3
20	Deque: Double-ended queue, implementation, applications (e.g., sliding window)	Implement dequeues to allow flexible insertion and deletion at both ends for applications like sliding window problems	TB1: Ch 10 Ref1: Ch 3
21	Trees: Binary trees, traversals (pre-order, in-order, post-order), applications (e.g., file systems)	Implement binary tree traversals to organize hierarchical data, such as file system structures	Ref1: Ch 4
Continued on next page			

Lecture no.	Description	Learning Outcomes	Reference
22-23	Binary Search Trees: Search, insert, delete, applications (e.g., dictionary lookup)	Build and manipulate binary search trees for ordered data storage, enabling dictionary lookup applications	TB1: Ch 12 Ref1: Ch 4
24	Balanced Search Trees: Need and overview	Understand the need for balanced search trees and identify common self-balancing tree structures.	TB1: Ch 12 Ref1: Ch 4
25	Hash Tables: Hash functions, collision resolution (chaining), applications (e.g., caching)	Build hash tables with chaining to handle collisions, enabling fast data retrieval for caching	TB1: Ch 11 Ref1: Ch 5
26	Hash Tables: Open addressing (linear probing), practical use cases	Implement hash tables with linear probing for collision resolution in applications like fast lookups	TB1: Ch 11 Ref1: Ch 5
27-28	Priority Queues: Binary heaps, insert, removeMin, applications (e.g., task prioritization)	Implement binary heaps for priority queues to manage prioritized tasks, such as event scheduling	TB1: Ch 6 Ref1: Ch 6
29-30	Heap Sort: Using heaps for sorting, applications (e.g., event scheduling)	Implement heap sort using binary heaps to sort data for applications like event prioritization	TB1: Ch 6
31	Graphs: Definitions, representations (adjacency list/matrix), examples (e.g., social networks)	Model real-world problems using graph representations, such as social network connections	TB1: Ch 22 Ref1: Ch 9
32-33	Graph Traversals: BFS, applications (e.g., shortest path in unweighted graphs), DFS, applications (e.g., cycle detection)	Implement BFS to find shortest paths in unweighted graphs, such as social network friend recommendations, Implement DFS to explore graph structures and detect cycles in applications like network analysis	TB1: Ch 22 Ref1: Ch 9
34-36	Greedy Algorithms: Concept, Minimum Spanning Tree: Algorithm, its correctness and applications (e.g., network design)	Apply greedy techniques like Kruskal's and Prim's algorithm to design efficient network structures	TB1: Ch 23
37-38	Single Source Shortest Path Algorithms: Bellman-Ford Algorithm and Dijkstra's algorithm, applications (e.g., GPS navigation)	Implement Dijkstra's algorithm to find shortest paths in weighted graphs for GPS navigation systems	TB1: Ch 24
39-40	Dynamic Programming: Concept, longest common subsequence, applications (e.g., plagiarism detection)	Apply dynamic programming to solve problems like longest common subsequence for plagiarism detection	TB1: Ch 15, 15.4

5.2 Laboratory Sessions Schedule

Lab Session No.	Topics
1	Warm-up Programming Session
2	Implement linear and binary search algorithms
3	Sorting: Implement Insertion and Merge Sort
4	Sorting: Implement quick sort and selection sort
5	Linked Lists: Implement singly and doubly linked lists
6	Stacks: Implement stack operations using arrays and linked lists
7	Queues and Dequeues: Implement queue operations using arrays and linked lists;
8	Trees: Implement tree traversal algorithms
9	Binary Search Trees: Implement search, insert, delete operations
10	Hash Tables: Implement chaining and linear probing
11	Priority Queues and Heaps: Implement binary heaps, insert, removeMin
12	Graphs: Implement adjacency lists, BFS, DFS and their application
13	MST: Implement Kruskal's algorithm for minimum spanning trees
14	Shortest Paths: Implement Dijkstra's SSSP

6. Evaluation Scheme:

Component	Wt (%) (Marks)	Date & Time	Duration	Comments
Mid-Semester	25% (25 M)	As per AUGSD schedule	90 mins	Closed Book
Lab Test	20% (20 M)	TBA	120 mins	Open Book
Comprehensive Exam	35% (35 M)	As per AUGSD schedule	180 mins	Partly Open Book
Lab Evaluation	20% (20 M)	Lab Sessions		Open Book
Total	100% (100 M)			

NOTE:

- Each lab session carries 2 marks each. For the final lab evaluation component, marks from the best 10 labs attended will be considered.
- **NC Criterion:** 30% of the average of the top three performers or 40% of median, whichever is lower, or a suitable modification at the discretion of the IC with the concurrence of the Department.

6.1 Open Book Policy

For open-book assessment components, students are permitted to bring their personal class notes, the prescribed textbook, and reference books. Solved previous-year question papers, solution manuals, or similar solved materials are not permitted.

6.2 Make-Up Policy

Make-ups are granted only for genuine reasons (e.g., medical issues) with prior permission:

- Lab Test: Email the Instructor-in-Charge with documentary proof.
- Mid-Term and Comprehensive Exam: Apply via the AUGSD portal.

Make-up requests must be submitted to the Instructor-in-Charge at least 24 hours prior to the scheduled exam. Please note that make-up exams are generally more challenging than the regular ones.

6.3 Academic Honesty and Integrity Policy

Academic honesty is mandatory. Any use of unfair means (e.g., copying, unauthorized sources) will be reported to AUGSD and dealt with strictly.

7. Office Consultation Hour:

Feel free to email me to seek a prior appointment.

Veena Prabhakaran	6121-E	Tuesday 11 AM–12 PM
-------------------	--------	---------------------

8. Notices

All notices will be posted on Nalanda.

Instructor-in-charge
Veena Prabhakaran
BITS F232